

Final Project

Laboratórios de Sistemas e Serviços

Mário Antunes

2026

Projects

This project is strictly individual. Select **one** of the following projects. All projects will be hosted on **GitHub**, using [GitHub Classroom](#). Check [here](#) for details.

The repository must contain all relevant scripts, configuration files, and a `README.md` with instructions on how to deploy the project. It should also contain a project report in PDF format.

This is the **final project**, designed to integrate the various skills acquired throughout the semester (Shell Scripting, Docker, Python, Data Analysis, and Web Technologies) from both **Project 01** and **Project 02**.

This is a three-week project (deadline **09/07/2026**). You have until the end of this week to notify your professor (via e-mail) of your chosen topic (the list of topics can be found [here](#)).

Do not forget to contact your professor with any questions. Further instructions may be added.

Topics

1. Enterprise IoT Monitoring Platform with Automated Backups & Caching

- **Description:** Deploy an industrial IoT monitoring pipeline. Multiple sensor simulator containers (Python scripts) publish synthetic environmental telemetry (temperature, humidity, vibration) via **MQTT**. A **Mosquitto** broker routes this telemetry. A **FastAPI** collector service subscribes to the MQTT topics, validates the incoming data, and stores it in a **PostgreSQL** database. **Grafana** is deployed to visualize the metrics in real-time. A **Caddy** (or Nginx) container serves as a caching reverse proxy in front of the Grafana dashboard. A **Bash**

backup script runs periodically via cron inside a dedicated backup container to export the PostgreSQL database, compress it into a .tar.gz file with a timestamp, and rotate old archives (retaining only the 7 most recent).

- **Core Skills:** Docker Compose (multi-container bridge network), MQTT (Mosquitto), SQL Persistence (PostgreSQL with volume mounts), Reverse Proxy & Caching (Caddy/Nginx), Bash Scripting & Automation (cron, tar, file rotation), Grafana.

2. Collaborative Task Board with Custom Dev Environments & Productivity Analytics

- **Description:** Create a collaborative real-time Kanban board. The web application allows multiple users to manage tasks, synchronizing card movements across clients instantly via **WebSockets**. The backend uses **Redis Pub/Sub** to manage message broadcast concurrency and stores all persistent task states in a **PostgreSQL** database. A separate, custom offline container runs a **Python script** using **Pandas** or **Polars** to read the database periodically, compute key metrics (such as the average cycle time per task and distribution of workloads), and save an analytical summary report. A **Bash daemon script** runs on the host (or a privileged container) to monitor the storage directory sizes, log database container resource usage, and append heartbeats to a system log. Developers must use a **Dev Container** setup (Dockerfile configuring debugging tools and linters) for local development.
- **Core Skills:** WebSockets, Redis Pub/Sub, PostgreSQL, Dev Containers (Dockerfile), Python Data Analytics (Pandas/Polars), Bash Daemon Scripting (system health and disk monitoring), Docker Compose.

3. Geo-Distributed Environmental Analytics Hub

- **Description:** Build a geographical dashboard that visualizes environmental metrics. A **Python service** periodically queries an **external REST API** (e.g., OpenWeatherMap, OpenAQ, or a public air quality API) for weather or environmental metrics in 5 pre-selected cities, persisting raw data in a database. A separate analytical **Python script** using **Pandas** runs to detect outliers, compute standard deviations (volatility), and export structured JSON data to a shared volume. An **Nginx** web server hosts a static web portal containing an **interactive Leaflet.js map** that reads the JSON data and displays color-coded markers based on air quality or weather severity. A **Bash script** runs on a timer to check the external API's availability (using `curl`), inspect the REST rate-limits, and append status logs.
- **Core Skills:** External REST APIs, Data Analysis (Pandas), Map Visu-

alization (Leaflet.js), Nginx Web Server, Bash Automation (curl API checking), Docker Compose.

4. Secure Multi-User Document & Media Vault

- **Description:** Implement a secure document library that automates metadata collection and cryptographic protection. A **FastAPI** web service handles authenticated file uploads (documents or media) and automatically queries an **external REST API** (e.g., TMDB or OpenLibrary) to scrape rich metadata based on the title, storing metadata in a database. When a new file is uploaded, a background **Bash script** acting as a directory watcher detects the file, verifies its integrity (SHA256 hashing), encrypts it using **GnuPG** or **OpenSSL** with a secure passphrase/key, and deletes the unencrypted original. The system must enforce strict **Linux file permissions** (chmod/chown in the volume) so that only the encryption agent can read raw files. A web frontend displays the metadata catalog and allows authorized users to request, decrypt, and stream files on the fly.
- **Core Skills:** Security Concepts (GnuPG/OpenSSL encryption, SHA256 hashing), Linux System Permissions (chmod/chown), FastAPI Uploads, Database persistence, External REST API scraping, Bash Directory Watching.

5. Database-Driven Markdown Report CI/CD Pipeline

- **Description:** Design an automated Continuous Integration pipeline that compiles analytical reports from database records. A multi-container application stores system metrics or e-commerce records in a **PostgreSQL** database. A **Bash script** monitors a PostgreSQL data view or log folder for updates. When a change is detected, it triggers a custom **Docker container** equipped with **Pandoc**, **LaTeX**, and **Python**. This container executes a Python script that runs queries, analyzes the database using **Pandas**, generates statistical plots (PNGs), updates a dynamic **Markdown** template, and compiles it via **Pandoc** into a final PDF report. The compiled PDF is automatically copied to a volume served by a **Caddy** web server, allowing users to download the latest report.
- **Core Skills:** Custom Dockerfiles (Pandoc + LaTeX + Python), Bash Automation (database/folder monitoring), Python Plotting (Matplotlib/Seaborn), Pandoc PDF compilation, SQL Databases, Caddy/Nginx web serving.

6. Smart Home Energy Management System with Technical Wiki

- **Description:** Create a smart home energy tracker supported by interactive wiki documentation. Simulators publish hourly home power consumption metrics via **MQTT** to a **Mosquitto** broker. A **FastAPI** collector stores this telemetry in **PostgreSQL**. **Grafana** is deployed to display energy trends and trigger alerts when thresholds are breached. To document the technical architecture, a **BookStack** or **DokuWiki** container is deployed alongside the system, pre-populated via a mounted volume with at least 5 markdown pages detailing container networking subnets, database schemas, and setup instructions. A **Bash script** runs periodically to check CPU and memory usage of the containers, logging the resource footprint.
- **Core Skills:** MQTT (Mosquitto), PostgreSQL, Grafana, Wiki Deployment (BookStack/DokuWiki), Docker Volumes & Networking, Bash Resource Telemetry.

GitHub Classroom Access

Here are detailed instructions to access GitHub Classroom. Most students can skip several steps, given that these were completed in previous projects.

1. Join the Assignment and Form Your Team

1. **Access the link:** Go to [here](#)
2. **Find your name:** Select your name from the student list. > **Can't find your name?** All names registered on PACO were added. If yours is missing, please contact [Prof. Mário Antunes](#).
3. **Create a team:** Follow this exact naming structure: [nmecc]_project03
 - (Example: 132745_project03)

2. Access the Organization and Repository

1. **Accept the email invite:** After joining a team, all members will receive an email invitation to join the detiuaveiro GitHub organization.
2. **You must accept this invitation** before you can continue.
3. **Refresh the page:** Go back to the GitHub Classroom page and refresh it.
4. **Verify access:** You should now see and have access to your team's working repository.

3. Configure an SSH Key for Access

This will allow you to clone and push to the repository from your command line without entering your password every time.

1. **Check for an existing SSH key:** Open your terminal and run this command:

```
cat ~/.ssh/id_ed25519.pub
```

2. **Generate a key (if needed):**

- If you see a key (starting with `ssh-ed25519 . . .`), copy the entire line and skip to step 3.
- If you see an error like “No such file or directory,” run the following command to create a new key:

```
ssh-keygen -q -t ed25519 -N ''
```

- After it's generated, run `cat ~/.ssh/id_ed25519.pub` again to view your new key and copy it.

3. **Add the key to your GitHub account:**

- Go to your GitHub **Settings**.
- On the left menu, click **SSH and GPG keys**.
- Click the **New SSH key** button.
- Give it a **Title** (e.g., “My UA Laptop”).
- Paste the key you copied into the **Key** field.
- Make sure the “Key type” is set to **Authentication Key**.
- Click **Add SSH key**.

4. **Authorize the key for SSO:**

- After adding the key, find it in your list on the same page.
- Click **Configure SSO**.
- Select the **detiuaveiro** organization, fill in your login details, and grant access.